Autonomous Search and Rescue

Project B - IA712 Mobile Robotics, Final Project

Team RobotZ

Julien GIMENEZ   Hugo FANCHINI   Paul CINTRA   Yimou ZHANG

`https://github.com/YiZeems/autonomous-search-and-rescue`

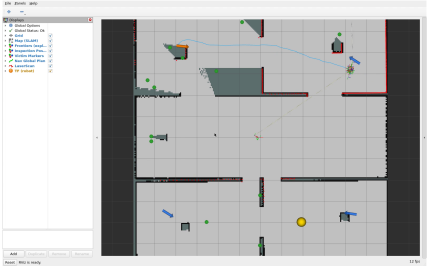
IA712 - Session L18

ROS 2 Humble · Ignition Gazebo Fortress · TurtleBot 4 · Nav2 · slam_toolbox · BehaviorTree.CPP

Scenario [1]: a robot explores an unknown disaster zone, finds "victims" (AprilTags), and reports their global coordinates - *no human input*.

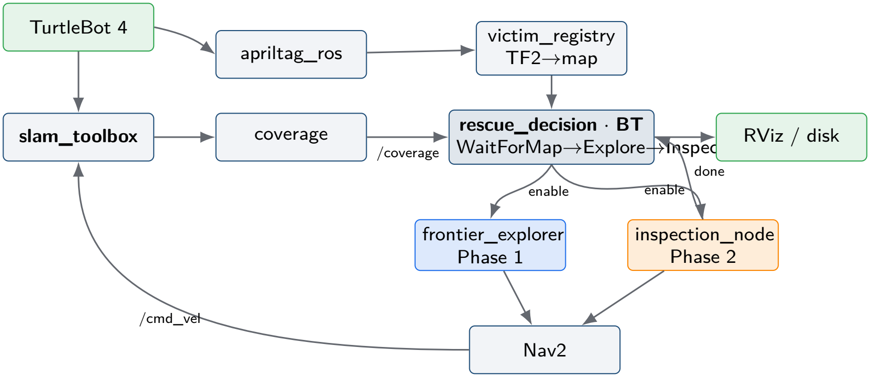
Requirements we satisfy:

- Autonomous frontier exploration, map $> 90\,\%$
- Camera detection $\rightarrow$ `map` via `tf2`
- Map quality / loop closure (SLAM)
- **Decision = Behavior Tree, not FSM**
- One-click launch
- Bonus: greedy vs. information-gain

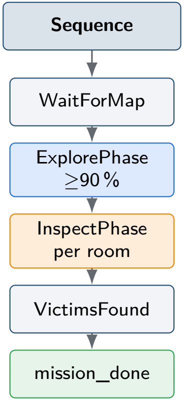


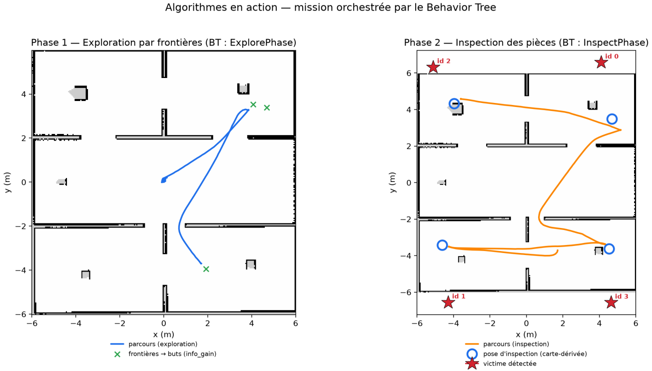
Live RViz: map + 4 victims + algorithms.

Result: one continuous autonomous run - **97.17 % coverage, 4/4 victims**.



The **Behavior Tree orchestrates**: it gates the explorer and the inspector via `/mission/*` topics; the Python nodes do the work.

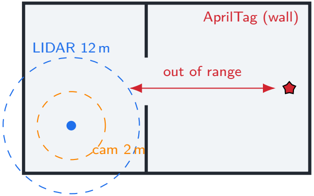




Phase 1 (blue) frontier exploration builds the map; **Phase 2** (orange) inspects each room at *map-derived* poses (no victim coordinates) to catch every wall tag.

Why two phases? Pure exploration maps rooms from doorways but the camera never gets within 2 m of the wall tags ⇒ it caps at ∼2 victims.

- **SLAM + loop closure** - consistent online map (CM6–7) [4].
- **Frontier exploration** - free/unknown boundary [2]; *information-gain* $g(f) = \text{gain} - \lambda\,\text{cost}$ [3] (CM8). Blacklist for dead frontiers.
- **Path planning** - Dijkstra/NavFn vs. A*; NavFn kept (tolerant in tight space) (CM9).
- **TF2** - project camera detection $\rightarrow$ global `map` (CM, requirement [8]).
- **Behavior Tree** - reactive, composable; memory `Sequence`, not an FSM [6].



The camera-vs-LIDAR gap that the inspection phase closes.

Run 1 - pure exploration
v20–v22

Blocked: $\sim$2 victims (camera $>$2 m from tags); loops on dead frontiers.

Improved: frontier blacklist; RTF knob; $\sim$90 % map.

Run 2 - two-phase + BT
v32

Blocked: completeness $\neq$ camera coverage; rigid patrol fragile.

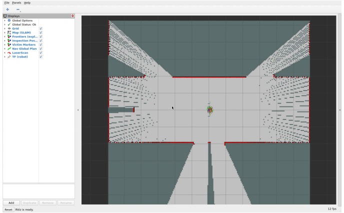
Improved: map-derived inspection; **BT orchestrates $\Rightarrow$ 4/4 + 97 %**.

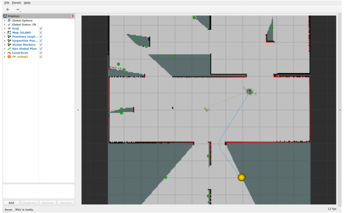
Run 3 - final
v37

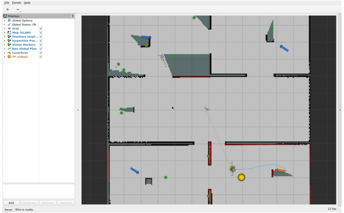
Blocked: RTF 1.5 $\rightarrow$ 3/4 + GPU stress; figures looked wall-crossing.

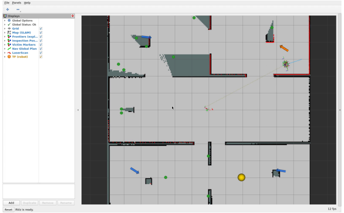
Improved: keep RTF 0.7; walls over path; HD capture.

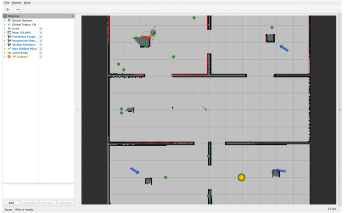
Two hidden root causes: Nav2 cost discouraged *entering* rooms (coverage); a collapsed RTF *starved the camera* (detection). The 2-phase BT then made 4/4 reliable.

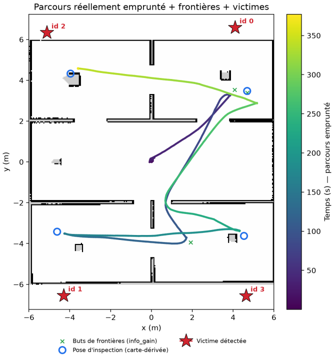


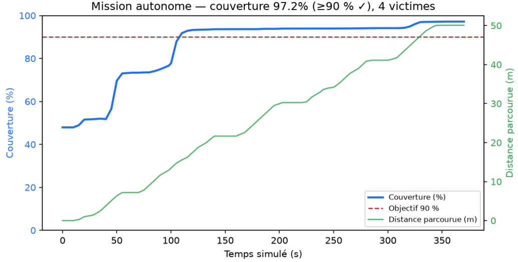




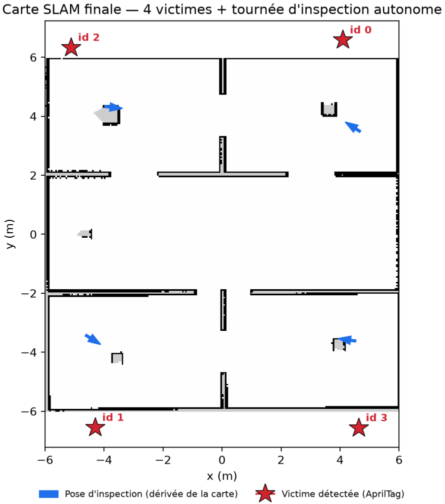





Parcours réellement emprunté + frontières + victimes
id 0
id 1
id 2
id 3
x (m)
y (m)
Temps (s) — parcours emprunté
Buts de frontières (info_gain)
Pose d'inspection (carte-dérivée)
Victime détectée



Coverage & path vs. time, 90 % target.

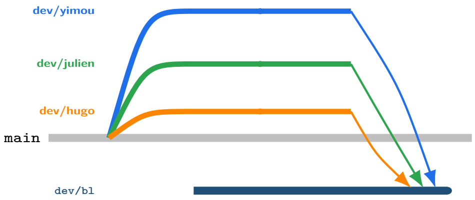


Final map: 4 victims + autonomous inspection tour.

97.17 % coverage · 4/4 victims · annotated map · logged trajectory - one click, fully autonomous.

Per-member `dev/*` branches merged into `dev/bl` (integration):

- **Yimou** - exploration, SLAM/Nav2, perception core
- **Julien** - BT decision, infra (WSL/GPU/RTF), results, integration
- **Hugo** - world / arena / AprilTag targets
- **Paul** - perception/detection support, testing



Three workstreams → `bl` integration.

- **Measure before prescribing** - "obvious" causes (memory, Windows Update) were wrong twice; event logs found the real ones (GPU TDR, a starved camera).
- **One fix reveals the next** - victim detection crossed six stacked causes.
- **Robust beats rigid** - adaptive exploration + 360° sweep beat a fixed patrol; the inspector also recovers a pose another machine cannot reach (wait for the costmap, fall back into the room) - *cross-machine robustness*.
- **Conformity by design** - a BT that *orchestrates* (not just supervises) meets the assignment in spirit, not only in letter.

Timeline: L13 kick-off · L14 architecture · L15 nodes/SLAM · L16 BT+perception · L17 integration/benchmark · L18 two-phase, final run, report.

Thank you - Questions?

We warmly thank **Zhi Yan**, Teacher-Researcher at ENSTA,
for the IA712 course and guidance.

https://github.com/YiZeems/autonomous-search-and-rescue

Julien GIMENEZ · Hugo FANCHINI · Paul CINTRA · Yimou ZHANG